

FORTRAN 90

ESTRUCTURAS DE CONTROL



VARIABLE LOGICAL

- Una variable tipo **LOGICAL** puede contener solamente dos valores **.TRUE.** o **.FALSE.**
Verdadero o falso.
- Fortran 90 tiene seis operadores relacionales:
<, <=, >, >=, ==, /=
 - Cada uno de estos operadores relacionales toma dos expresiones, compara los valores y devuelve **.TRUE.** o **.FALSE.**
 - Los valores **COMPLEX** pueden usar solamente **==** y **/=**
 - Los valores **LOGICAL** pueden ser **.EQV., .NEQV., .NOT., .OR., .AND.,** para comparación.

DECLARACIÓN IF-THEN-ELSE

- Fortran tiene tres formas **IF-THEN-ELSE**
- La más completa es **IF-THEN-ELSE-IF-END IF**
- También la declaración **IF** sola puede ser útil.
- Las expresiones lógicas se evalúan secuencialmente, es decir de arriba para abajo, La secuencia de instrucciones que corresponda a la expresión evaluada para **.TRUE.** será ejecutada.
- De otra manera la secuencia **ELSE** será ejecutada.

```
IF (expresion-logica-1) THEN
    secuencia de declaraciones 1
ELSE IF (expresion-logica-2) THEN
    secuencia de declaraciones 2
ELSE IF (expresion-logica-3) THEN
    secuencia de declaraciones 3
ELSE IF (.....) THEN
    .....
ELSE
    secuencia de declaraciones
END IF
```

DECLARACIÓN IF-THEN-ELSE

- Dos ejemplos:

Encuentra el valor mínimo entre a, b y c y guarda el resultado en Result:

```
IF (a < b .AND. a < c) THEN
    Result = a
ELSE IF ( b < a .AND. b < c) THEN
    Result = b
ELSE
    Result = c
END IF
```

Calificación por letra:

```
IF (x < 50) THEN
    Calif = 'NA'
ELSE IF (x < 70) THEN
    Calif = 'S'
ELSE IF (x < 80) THEN
    Calif = 'R'
ELSE IF (x < 90) THEN
    Calif = 'B'
ELSE
    Calif = 'MB'
END IF
```

DECLARACIÓN IF-THEN-ELSE

- La declaración **ELSE-IF** y la declaración **ELSE** son opcionales.
- Si la declaración **ELSE** no existe y ninguna de las expresiones lógicas resulta **.TRUE.**, la declaración **IF-THEN-ELSE** no tiene efecto.

Sin **ELSE-IF**

```
IF (expresion-logica-1) THEN
    secuencia de declaraciones 1
ELSE
    secuencia de declaraciones ELSE
END IF
```

Sin **ELSE**

```
IF (expresion-logica-1) THEN
    secuencia de declaraciones 1
ELSE IF (expresion-logica-2) THEN
    secuencia de declaraciones 2
ELSE IF (expresion-logica-3) THEN
    secuencia de declaraciones 3
ELSE IF (.....) THEN
    .....
END IF
```

EJEMPLO

- Dada una ecuación cuadrática $ax^2 + bx + c = 0$, donde $a \neq 0$ sus raíces se calculan de la siguiente manera:

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

```
PROGRAM EcuacionCuadratica
IMPLICIT NONE
```

```
REAL :: a, b, c
```

```
REAL :: d
```

```
REAL :: raiz1, raiz2
```

```
READ a, b, c
```

```
WRITE(*,*) 'a = ', a
```

```
WRITE(*,*) 'b = ', b
```

```
WRITE(*,*) 'c = ', c
```

```
WRITE(*,*)
```

```
d = b**2 - 4.0*a*c
```

```
IF (d >= 0.0) THEN      !¿Tiene solucion?
```

```
    d = SQRT(d)
```

```
    raiz1 = (-b + d)/(2.0*a) ! Raiz 1
```

```
    raiz2 = (-b - d)/(2.0*a) ! Raiz 2
```

```
WRITE(*,*) 'Las raices son', raiz1, ' &  
y ', raiz2
```

```
ELSE                      ! Raices complejas
```

```
WRITE(*,*) 'Na hay raices reales'
```

```
WRITE(*,*) 'Discriminante = ', d
```

```
END IF
```

```
END PROGRAM EcuacionCuadratica
```

EJEMPLO

- Si anexamos el caso de raíces repetidas.

```
IF (d >= 0.0) THEN      !¿Tiene solucion?
  d = SQRT(d)
  raiz1 = (-b + d)/(2.0*a) ! Raiz 1
  raiz2 = (-b - d)/(2.0*a) ! Raiz 2
  WRITE(*,*) 'Las raices son', raiz1, ' y ', raiz2
  !-----

ELSE IF (d == 0.0) THEN  !Raices repetidas
  WRITE(*,*) 'La raiz repetida es', -b/(2.0*a)

  !-----
ELSE                      ! Raices complejas
  WRITE(*,*) 'Na hay raices reales'
  WRITE(*,*) 'Discriminante = ', d
END IF
```

IF LÓGICO

- Cuando el IF lógico devuelve un valor .TRUE. La declaración es ejecutada de lo contrario el programa sigue su curso.

`IF (expresion-logica) declaracion`

- Por ejemplo:

```
ElMenor = b  
IF (a < b) ElMenor = a
```


DECLARACIÓN SELECT CASE

- Fortran 90 tiene la declaración **SELECT CASE** para la ejecución selectiva si el criterio de selección está basado en valores simples, **INTEGER**, **LOGICAL**, y **CHARACTER**. No es aplicable para valores **REAL**.

```
SELECT CASE (selector)
  CASE (etiqueta-1)
    declaraciones-1
  CASE (etiqueta-2)      !selector es una expresion que evalua una
    declaraciones-2      !variable INTEGER, LOGICAL o CHARACTER
  CASE (etiqueta-3)      !etiqueta es un paquete de constantes o
    declaraciones-3      !parametros del mismo tipo que el selector

!.....otros casos
  CASE (etiqueta-n)
    declaraciones-n
CASE DEFAULT
  declaraciones-POR DEFECTO
END SELECT
```

DECLARACIÓN **SELECT CASE**

- Las etiquetas pueden tener las siguientes formas:
 - `valor1` – busca un valor específico
 - `valor1 : valor2` – valores entre `valor1` y `valor2`, incluyendo `valor 1` y `valor 2`, y `valor1` menor o igual que `valor 2`
 - `valor1 :` – valores más grandes o iguales que `valor 1`
 - `:` `valor2` – valores menores o iguales a `valor2`
- La declaración **SELECT CASE** se ejecuta de la siguiente forma:
 - Compara el valor del selector con las etiquetas de cada caso. Si existe una concordancia, se ejecutan las declaraciones correspondientes a ese caso.
 - Si no se encuentra una concordancia se ejecuta el caso **DEFAULT** o **CASE DEFAULT**

EJEMPLO

- Este programa lee un caracter y determina si es una vocal, una consonante, un digito, alguno de los cuatro operadores aritméticos, un espacio en blanco, o algún otro tipo de caracter.

```
PROGRAM PruebaCaracter
IMPLICIT NONE
CHARACTER(LEN=1) :: Entrada
READ(*,*) Entrada
SELECT CASE (Entrada)
  CASE ('A' : 'Z', 'a' : 'z')           ! Descarta las letras
    WRITE(*,*) 'Se encontro una letra : "', Entrada, '"'
    SELECT CASE (Entrada)               ! ¿Una vocal?
      CASE ('A', 'E', 'I', 'O', 'U', 'a', 'e', 'i', 'o', 'u')
        WRITE(*,*) 'Es una vocal'
      CASE DEFAULT                       ! Debe ser una consonante
        WRITE(*,*) 'Es una consonante'
    END SELECT
  CASE ('0' : '9')                       ! Un digito
    WRITE(*,*) 'Se encontro un digito : "', Entrada, '"'
  CASE ('+', '-', '*', '/')              ! Un operador
    WRITE(*,*) 'Se encontro un operador : "', Input, '"'
  CASE (' ')                             ! Espacio en blanco
    WRITE(*,*) 'Se encontro un espacio en blanco : "', Entrada, '"'
  CASE DEFAULT                           ! Caracter especial
    WRITE(*,*) 'Se encontro otro tipo de caracter : "', Entrada, '"'
END SELECT
END PROGRAM PruebaCaracter
```

EL BUCLE DO

- Fortran 90 tiene dos formas de bucle **DO**: El conteo **DO** y **DO** general.
- El **DO** de conteo tiene la siguiente forma:

```
DO var-control = inicial, final [, step]
    declaraciones
END DO
```

- **var-control** es una variable **INTEGER**, **inicial**, **final** y **step** son expresiones **INTEGER**; Sin embargo, **step** no puede ser cero.
- Si se omite el **step**, su valor por defecto es 1.
- Antes que un bucle-**DO** comience, las expresiones **inicial**, **final** y **step** son evaluadas solamente una vez. Cuando se ejecute el bucle-**DO**, estos valores no serán reevaluados.
- Si **step** es positivo, el **DO** incrementa la cuenta y si **step** es negativo, el **DO** decrementa la cuenta.

EL BUCLE DO

- Si **step** es positivo:
 - La **var-control** recibe el valor de **inicial**.
 - Si el valor de **var-control** es menor o igual que el valor de **final** la parte de declaraciones es ejecutada. Después, se le agrega el valor de **step** a **var-control**, y vuelve a comparar los valores de **var-control** y **final**.
 - Si el valor de **var-control** es mayor que el valor de **final** el bucle-DO se completa y la declaración **END DO** es ejecutada.
- Si **step** es negativo:
 - La **var-control** recibe el valor de **inicial**.
 - Si el valor de **var-control** es mayor o igual que el valor de **final** la parte de declaraciones es ejecutada. Después, se le agrega el valor de **step** a **var-control**, y vuelve a comparar los valores de **var-control** y **final**.
 - Si el valor de **var-control** es menor que el valor de **final** el bucle-DO se completa y la declaración **END DO** es ejecutada.

EL BUCLE DO

- Dos ejemplos:

```
INTEGER :: N, k                                ! Enteros nones entre 1 y N
```

```
READ(*,*) N  
WRITE(*,*) "Numero non entre 1 y ", N  
DO k = 1, N, 2  
    WRITE(*,*) k  
END DO
```

```
INTEGER, PARAMETER :: LONG = SELECTED_INT_KIND(15)    ! Factorial de N  
INTEGER(KIND=LONG) :: Factorial, i, N
```

```
READ(* *) N  
READ(*,*) Factorial = 1_LONG  
DO i = 1, N  
    Factorial = Factorial * i  
END DO  
WRITE(*,*) N, "! = ", Factorial
```

EL BUCLE DO CON EXIT

- El bucle-DO tiene la siguiente forma:

DO

Declaraciones

END DO

- Las declaraciones serán ejecutadas repetidamente.
- Para salir del bucle-DO, se usa EXIT o la declaración CYCLE.
- La declaración EXIT lleva al flujo de control a la declaración END DO.
- La declaración CYCLE comienza la siguiente iteración, ejecutando las declaraciones de nuevo.
- Por ejemplo:

```
REAL, PARAMETER :: Menor = -1.0, Mayor = 1.0, Paso = 0.25
REAL :: x
x = Menor ! inicializando la variable de control
DO
  IF (x > Mayor) EXIT ! es > valor-final?
  WRITE(*,*) x ! no, haz el bucle
  x = x + Paso ! Incrementa por tamaño de paso
END DO
```

EJEMPLO

- La función $\exp(x)$ tiene una serie infinita:

$$\exp(x) = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \cdots + \frac{x^n}{n!}$$

- Suma cada termino hasta que el valor absoluto del termino es menor que una tolerancia, digamos 0.00001.
- Nota:

$$\frac{x^{i+1}}{(i+1)!} = \left(\frac{x^i}{i!}\right) \left(\frac{x}{i+1}\right)$$

EJEMPLO

```
PROGRAM Exponencial
  IMPLICIT NONE
  INTEGER :: Cuenta                                ! # de terminos utilizados
  REAL :: Term                                     ! Un termino
  REAL :: Sum                                       ! La suma
  REAL :: X                                         ! La entrada x
  REAL, PARAMETER :: Tolerancia = 0.00001         ! tolerancia

  READ(*,*) X                                       ! lee x
  Cuenta = 1                                       ! El primer termino es 1
  Sum = 1.0                                       ! Por lo tanto, la suma comienza con 1
  Term = X                                         ! El segundo termino es x
  DO                                               ! Para cada termino
    IF (ABS(Term) < Tolerancia) EXIT              ! Si es muy pequeña, salir
    Sum = Sum + Term                              ! Si no, agrega a la suma
    Cuenta = Cuenta + 1                          ! Cuenta indica el siguiente termino
    Term = Term * (X / Cuenta)                   ! Calcula el valor del siguiente termino
  END DO
  WRITE(*,*) 'Depues de ', Cuenta, ' iteraciones:'
  WRITE(*,*) ' Exp(', X, ') = ', Sum
  WRITE(*,*) ' De la funcion EXP() = ', EXP(X)
  WRITE(*,*) ' Abs(Error) = ', ABS(Sum - EXP(X))
END PROGRAM Exponencial
```